

Manuale dello Sviluppatore — SkillShare

Indice:

1. Introduzione e scopo del documento
2. Requisiti di Sistema e prerequisiti
3. Guida all'installazione ed all'avvio
4. Architettura e Tecnologie
5. Panoramica delle Funzionalità Realizzate
6. Design Pattern utilizzati
7. Credenziali di accesso demo

1. Introduzione e Scopo del Documento

1.1 Descrizione del Progetto: Skillshare

Skillshare è una piattaforma web peer-to-peer finalizzata allo scambio diretto di competenze tra utenti. A differenza dei modelli tradizionali il sistema si struttura invece come un'**economia del baratto**. All'interno dell'applicazione, ogni utente riveste contemporaneamente il duplice ruolo di **studente** e **insegnante**, permettendo la condivisione diretta di conoscenze che spaziano dagli ambiti tecnici (come lo sviluppo software) a quelli pratici (come la cucina o lo studio delle lingue).

1.2 Scopo del Documento

Il presente documento è redatto come guida tecnica ufficiale rivolta ai membri del team e a eventuali collaboratori esterni che desiderano contribuire al progetto, agevolando tali attraverso:

- La configurazione dei prerequisiti di sistema e l'installazione dell'ambiente di sviluppo locale.
- I flussi di avvio dell'applicazione tramite Maven e l'isolamento dell'ambiente in container Docker.
- La comprensione dettagliata dell'architettura software del progetto, analizzando la suddivisione dei moduli GWT (Client, Server, Shared), la logica di business e i meccanismi di persistenza dei dati.

2. Requisiti di Sistema e Prerequisiti

Prima di iniziare a lavorare sul progetto, un nuovo collaboratore deve assicurarsi che i seguenti strumenti siano installati e configurati correttamente sul proprio computer.

2.1 Tabella degli Strumenti Richiesti

Strumento	Versione Richiesta	Scopo nel Progetto
Java Development Kit (JDK)	17 o superiore	Ambiente di runtime e compilazione del codice Java.

Apache Maven	3.8+	Gestione delle dipendenze, automazione della build e dei test.
Docker & Docker Compose	Ultima versione stabile	Isolamento dell'ambiente, gestione del database e deploy in container.
Git	Ultima versione stabile	Controllo di versione e collaborazione sul codice tramite GitHub.
IDE Consigliato	A scelta (v. sotto)	Ambiente di sviluppo locale per la scrittura del codice.

2.2 Guida all'Installazione dei Componenti Core

I comandi aggiornati e le procedure dettagliate passo-passo per l'installazione del JDK 17 e di Maven sui diversi sistemi operativi (Ubuntu/Debian e Windows) sono disponibili e mantenuti direttamente nel file **README.md** situato nella cartella principale del progetto (/app/README.md o nella root della repository).

2.3 Ambiente di Sviluppo Consigliato

Il team non impone un unico editor, ma l'IDE deve essere configurato per supportare correttamente progetti **Maven multi-modulo** e il framework **GWT (Google Web Toolkit)**, quella che viene consigliata è **Visual Studio Code** in quanto Leggero e personalizzabile, richiede l'estensione Extension Pack for Java e Maven for Java.

3. Guida all'Installazione e Avvio

Nota per lo sviluppatore: I comandi completi, la sintassi esatta e le opzioni di debug per l'avvio e il testing dell'applicazione sono documentati in modo esaustivo nel file **README.md** presente nella cartella principale del progetto (/app/README.md).

Di seguito viene descritto il flusso logico delle operazioni che ogni nuovo collaboratore deve seguire per configurare ed eseguire l'applicazione in locale.

3.1 Compilazione del Progetto

Prima di eseguire l'applicazione, è necessario posizionarsi tramite terminale nella cartella principale (/app) e lanciare la build di Maven (**mvn clean install**).

3.2 Esecuzione in Locale (Modalità Sviluppo)

L'architettura di Skillshare richiede l'esecuzione di due processi simultanei, da lanciare in **due terminali distinti** all'interno della cartella di root:

1. **GWT Code Server (Frontend):** gestisce la compilazione del codice lato client.
2. **Server Jetty (Backend):** gestisce la compilazione del codice lato server e shared.

Una volta avviati entrambi i servizi, l'interfaccia utente è raggiungibile nel browser all'indirizzo standard (<http://127.0.0.1:8080/>).

Il progetto include test automatizzati JUnit che possono essere lanciati interamente o targettizzando un singolo modulo (es. il server) o una specifica classe, utile per verificare la correttezza del codice prima di effettuare un commit. I comandi di filtraggio dei test sono disponibili nel README.md

4. Architettura e Tecnologie

Il codice è organizzato in tre moduli logici (client, server, shared) che cooperano secondo il flusso descritto di seguito.

4.1 Front-end (Modulo Client)

L'interfaccia utente della piattaforma viene eseguita interamente all'interno del browser dell'utente, ma è scritta interamente in Java.

- **Compilazione Java-to-JavaScript:** tramite il compilatore GWT, che traduce il codice sorgente Java in file JavaScript pronti per l'esecuzione lato client senza che ci sia bisogno per lo sviluppatore di saper scrivere codice JavaScript o HTML.
- **UI a Componenti:** ogni vista (la schermata di login, il pannello del profilo, la bacheca delle competenze, ecc...) è un oggetto Java autonomo e riutilizzabile, che incapsula sia la struttura grafica che la gestione degli eventi utente.

4.2 Back-end (Modulo Server)

Il motore lato server gestisce il nucleo della logica di business dell'applicazione, il controllo degli accessi e la persistenza dei dati.

- **Chiamata al Server:** la comunicazione tra il client e il server non avviene tramite classiche API REST scritte a mano, ma sfrutta il meccanismo nativo **GWT RPC**. Questo protocollo consente al modulo client di invocare metodi Java esposti dal server come se fossero funzioni locali. Il framework gestisce automaticamente la serializzazione dei dati, garantendo un controllo rigoroso dei tipi tra client e server ed eliminando il rischio di errori.

4.3 Modulo Shared

Questo modulo fa da "collante" ed è visibile sia al client che al server, contiene:

- I **DTO**: ovvero i modelli dei dati che transitano nella rete (es. l'oggetto Utente o Annuncio).
- Le interfacce dei servizi che definiscono quali metodi il client può invocare sul server.

4.4 Persistenza e Database

Il sistema si affida a **MapDB**, un motore di database scritto interamente in Java. MapDB fornisce strutture dati familiari (come mappe e set transazionali) che vengono registrate e sincronizzate direttamente su un file locale nel disco rigido: **progetto_sweng.db**.

Tutta la logica di scrittura, lettura e commit sul file di database è centralizzata all'interno della classe **DatabaseCore**, mentre la logica di inizializzazione è gestita separatamente dalla classe **DatabaseSeeder**.

5. Panoramica delle Funzionalità Realizzate

5.1 Meccanismi di Funzionamento

- **Layout Dinamico:** per evitare il ricaricamento dell'intera pagina del browser, l'interfaccia utente adotta un approccio dinamico gestito dalla classe `MainLayoutGui`. Questa classe definisce una struttura fissa (l'header con il menu di navigazione) e ospita un `SimplePanel` chiamato `contenitoreDinamico`. Tramite il metodo `cambiaVista(Widget nuovaVista)`, il pannello svuota il proprio contenuto e vi inserisce a caldo la nuova schermata richiesta dall'utente (es. `MarketGui`, `ChatGui`), garantendo una navigazione fluida ed immediata.
- **Popolamento Automatico (Database Seeder):** La classe `DatabaseSeeder` genera a caldo un set completo di entità di test (utenti, annunci, messaggi e valutazioni), garantendo un ambiente di sviluppo e testing immediatamente pronto all'uso. Gli sviluppatori possono personalizzare lo scenario demo aggiungendo, rimuovendo o modificando le istanze direttamente all'interno del codice di questa classe.
- **Gestione della Sessione (SessionManager):** Per mantenere lo stato dell'utente autenticato senza ricaricare la pagina, l'applicazione centralizza la sessione nella classe `SessionManager`, che fornisce metodi globali per verificare se una sessione è attiva, recuperare i dati dell'utente corrente per personalizzare le viste dinamiche e coordinare in modo sicuro le operazioni di login e logout.

5.2 Struttura del Modulo Client (skillshare-client)

Classe / Interfaccia	Descrizione Breve e Funzionalità
App.java	Entry Point dell'applicazione, inizializza la pagina, lancia il seeding iniziale, controlla lo stato della sessione e lancia la schermata iniziale.
LoginGui.java	Schermata di autenticazione per gli utenti già registrati.
RegisterGui.java	Interfaccia per la creazione di un nuovo account sulla piattaforma.
MainLayoutGui.java	Lo scheletro principale dell'app, gestisce la barra di navigazione e lo scambio dinamico delle viste nel <code>contenitoreDinamico</code> .
MarketGui.java	La bacheca pubblica in cui gli utenti possono visualizzare e cercare tutti gli annunci di skill offerti dalla community.
ForYouGui.java	Feed personalizzato che mostra annunci raccomandati all'utente in base alle sue categorie preferite.
CreateAdGui.java	Form guidato che permette a un utente di pubblicare un nuovo annuncio per offrire una propria competenza.

ChatGui.java	Interfaccia della messaggistica interna, permette agli utenti di accordarsi privatamente sui dettagli dello scambio.
RichiesteGui.java	Pannello di gestione dove l'utente monitora lo stato delle proprie proposte di baratto (inviate, accettate, rifiutate, concluse).
SkillGui.java	Area dedicata alla gestione e visualizzazione dei propri annunci e delle richieste relative ad essi nei vari stati (pubblicati, in attesa, accettati, rifiutati, concluse).
ProfileGui.java	Pagina del profilo personale contenente i dati anagrafici, i badge acquisiti e le valutazioni (recensioni) ricevute.
SessionManager.java	Gestore lato client dello stato dell'utente, memorizza localmente l'utente loggato per renderlo accessibile a tutte le pagine.

5.3 Struttura del Modulo Shared (skillshare-shared)

Classe / Interfaccia	Descrizione Breve e Funzionalità
*Service.java	È il contratto principale della comunicazione RPC. Definisce le firme dei metodi che il server espone al client.
*ServiceAsync.java	È l'interfaccia utilizzata esclusivamente dal client per effettuare le chiamate verso il server.
Annuncio.java	Rappresenta l'offerta di una skill pubblicata sul mercato (titolo, descrizione, categoria, ecc..).
RichiestaScambio.java	Incapsula la transazione di baratto tra due utenti, tracciando chi propone lo scambio, chi lo riceve, lo stato attuale e l'id dell'annuncio corrispondente.
Messaggio.java	Rappresenta la singola riga di testo scambiata nella chat, completa di mittente, destinatario e messaggio effettivo.
Valutazione.java	Gestisce il sistema di recensioni rilasciate dagli utenti al termine di uno scambio concluso e contiene i riferimenti (autore, destinatario, messaggio e voto).
Utente.java	Rappresenta l'iscritto alla piattaforma (credenziali, bio, foto e le competenze preferite).

FieldVerifier.java	Classe di utilità condivisa per validare l'input dei dati (es. controllo formato email o lunghezza password).
---------------------------	---

5.4 Struttura del Modulo Server (skillshare-server)

Classe / Interfaccia	Descrizione Breve e Funzionalità
*Impl.java	Queste classi fungono da controller, ricevendo le richieste dal client, eseguendo i controlli di logica di business e orchestrando le operazioni di lettura/scrittura sul database.
DatabaseCore.java	È il modulo centrale per la gestione della persistenza, gestisce le transazioni (commit/rollback) ed espone le mappe per l'accesso ai dati (utenti, annunci, messaggi, ecc.).
DatabaseSeeder.java	Si occupa di popolare il database con un set di dati di test (utenti, annunci, messaggi e richieste di scambio) nel caso in cui le mappe risultino vuote.
AppInitializer.java	Si attiva automaticamente all'avvio del web server Jetty, il suo compito è innescare le procedure di inizializzazione del database e il caricamento delle categorie, oltre a gestire correttamente la chiusura delle risorse quando l'applicazione viene arrestata.
*SeleniumTest.java	Suite di test automatizzati dedicati alla verifica del frontend, simulano le interazioni dell'utente finale nel browser per validare che l'interfaccia e i flussi di navigazione comunichino correttamente con il backend.
*ImplTest.java	Test unitari focalizzati sulle classi di implementazione, verificano la logica di business in isolamento, assicurando che i metodi di servizio elaborino i dati e interagiscano con il database secondo le specifiche.

6. Design Pattern Utilizzati

- **Pattern Singleton**
 - **Dove è usato:** Nelle classi **DatabaseCore** e **SessionManager**.
 - **Vantaggi:** Garantisce che esista una sola istanza della connessione al database in tutta l'applicazione, evitando conflitti di accesso e lock sui file. Inoltre, mantiene uno stato globale centralizzato per l'utente loggato.
- **Pattern Builder**
 - **Dove è usato:** Nelle classi **Annuncio**, **CreateAdGui**, **Valutazione** che contengono una classe statica nidificata chiamata **Builder**.
 - **Vantaggi:**

- **Leggibilità:** elimina la confusione dei costruttori con troppi parametri in riga, esplicitando il nome di ogni campo configurato.
- **Flessibilità:** gestisce in modo pulito classi complesse o con molti attributi (come **Annuncio**, che ha ben 7 campi).
- **Stato Coerente:** Impedisce la creazione di oggetti parziali.

7. Credenziali di Accesso Demo

Note sull'Inizializzazione dei Dati

- **Controllo di Esistenza:** La procedura di seeding viene eseguita da **DatabaseCore** solo se le mappe degli utenti nel database risultano vuote: se il file fisico del database (**progetto_sweng.db**) esiste già, il sistema salta il seeding per preservare i dati correnti.
- **Procedura di Reset:** Se desideri ripristinare i dati demo allo stato originale o applicare modifiche strutturali apportate a **DatabaseSeeder**, devi **eliminare manualmente il file del database (progetto_sweng.db)** prima di riavviare l'applicazione.
- **Inizializzazione degli utenti:** admin è inizializzato senza nessun contenuto interno, mentre gli altri utenti hanno già annunci, valutazioni, chat e richieste nei vari stati.

Username	Password
filker67	password
brung05	password
leonisi007	password
mastroky	password
admin	password